Белорусский государственный университет

Факультет прикладной математики и информатики

Лабораторная работа №1

Цветовые модели: RGB, CMYK, HSL

Выполнил:

студент 3 курса

группы 4б

Гончаренко Артём

Минск, 2026

Содержание

1 Введение **2**

2 Стек технологий **2**

3 Математические основы преобразований **3**

3.1 RGB → CMYK 3

3.2 CMYK → RGB 3

3.3 RGB → HSL 3

3.4 HSL → RGB 4

4 Архитектура приложения **4**

4.1 Первый вариант архитектуры и его проблема 4

4.2 Исправленная архитектура 5

4.3 Паттерн «Наблюдатель» 6

5 Пользовательский интерфейс **6**

5.1 Отрисовка палитр 7

6 Возникшие проблемы и их решения **7**

6.1 Проблема необратимости преобразований 7

6.2 Дёргание маркера на HSL-палитре 8

6.3 Согласование отображения после пересчёта 8

7 Выводы **8**

1. Введение

Цель работы: изучить цветовые модели RGB, CMYK и HSL, реализовать взаимные преобразования между ними, а также разработать интерактивное приложение, позволяющее пользователю задавать цвет тремя различными способами в каждой из трёх моделей с автоматическим пересчётом компонент во всех остальных представлениях.

Задачи:

1. Разобрать математические формулы преобразований RGB $\leftrightarrow$ CMYK, RGB $\leftrightarrow$ HSL, а также преобразований между CMYK и HSL через RGB.

2. Разработать архитектуру приложения, обеспечивающую согласованное хранение состояния цвета.

3. Реализовать интерфейс с тремя способами задания цвета для каждой модели: палитра (клик и перетаскивание), ползунки, поля точного ввода.

4. Обеспечить автоматический пересчёт цвета во всех моделях при изменении любой из компонент.

2. Стек технологий

Для реализации приложения использовались следующие технологии:

- **HTML5** — разметка пользовательского интерфейса, включая элемент `<canvas>` для отрисовки цветовых палитр и элементы управления `<input type='range'>` и `<input type='color'>`.

- **CSS3** — оформление интерфейса: сеточная раскладка (`CSS Grid`), адаптивность через медиазапросы, кастомные стили для ползунков и вспомогательных элементов.

- **JavaScript (ECMAScript 2015+, ES6)** — вся логика приложения: преобразования цветовых моделей, обработка событий пользователя,

отрисовка палитр на canvas, синхронизация состояния. Использованы возможности ES6: `let`/`const`, стрелочные функции, шаблонные строки, деструктуризация, дефолтные параметры.

Приложение работает полностью на стороне клиента, без серверной части и без внешних библиотек. Вся работа ведётся с DOM-элементами и объектом `CanvasRenderingContext2D`.

3. Математические основы преобразований

3.1. RGB → CMYK

Значения R, G, B нормируются к диапазону $[0, 1]$. Тогда:

$$K = 1 - \max(R, G, B) \tag{1}$$

$$C = \frac{1 - R - K}{1 - K}, \quad M = \frac{1 - G - K}{1 - K}, \quad Y = \frac{1 - B - K}{1 - K} \tag{2}$$

При $K = 1$ (чистый чёрный цвет) компоненты C, M, Y полагаются нулевыми, поскольку восстановить их из чёрного невозможно.

3.2. CMYK → RGB

$$R = (1 - C)(1 - K), \quad G = (1 - M)(1 - K), \quad B = (1 - Y)(1 - K) \tag{3}$$

3.3. RGB → HSL

Пусть $\max = \max(R, G, B)$, $\min = \min(R, G, B)$, $\Delta = \max - \min$. Тогда:

$$L = \frac{\max + \min}{2} \tag{4}$$

$$S = \begin{cases} 0, & \Delta = 0, \\ \dfrac{\Delta}{\max + \min}, & L \le 0.5, \\ \dfrac{\Delta}{2 - \max - \min}, & L > 0.5. \end{cases} \tag{5}$$

Оттенок H вычисляется отдельно для каждого доминирующего канала:

$$H = \begin{cases} 60 \cdot \left(\dfrac{G - B}{\Delta} \bmod 6 \right), & \max = R, \\ 60 \cdot \left(\dfrac{B - R}{\Delta} + 2 \right), & \max = G, \\ 60 \cdot \left(\dfrac{R - G}{\Delta} + 4 \right), & \max = B. \end{cases} \tag{6}$$

3.4. HSL → RGB

Промежуточные величины:

$$C = (1 - |2L - 1|) \cdot S, \qquad X = C \cdot \left(1 - \left|\left(\tfrac{H}{60} \bmod 2\right) - 1\right|\right), \tag{7}$$

$$m = L - \frac{C}{2}. \tag{8}$$

Далее в зависимости от сектора H формируется промежуточная тройка (R', G', B'), и итоговые значения получаются как $R = R' + m$ и т.д.

4. Архитектура приложения

Приложение построено по принципу **единственного источника истины для каждой модели** и разделения ответственности между подсистемами хранения состояния и отрисовки.

4.1. Первый вариант архитектуры и его проблема

Изначально состояние цвета хранилось только в виде трёх целочисленных значений RGB. Все остальные модели считались производными: любое изменение пользователем CMYK- или HSL-компонент приводило к пересчёту в RGB, а затем — к обратному пересчёту в исходную модель.

Такая схема оказалась неработоспособной в окрестностях **необратимых точек**:

- при $K = 100$ в CMYK любые значения C, M, Y дают один и тот же чёрный цвет $(0, 0, 0)$ в RGB; при обратном пересчёте RGB → CMYK компоненты C, M, Y теряются, и ползунки «отскакивают» в ноль;

- аналогично при $L = 0$ или $L = 100$ в HSL любые H и S дают один и тот же цвет, и при обратном пересчёте оттенок и насыщенность теряются;

- на границах HSL-треугольника маркер текущего цвета «дёргался» из-за накопления ошибок округления при цепочке HSL $\rightarrow$ RGB (целые) $\rightarrow$ HSL.

Таким образом, единая точка хранения в RGB приводила к потере информации, вносимой пользователем в исходной цветовой модели.

4.2. Исправленная архитектура

Было принято решение хранить **три независимых состояния** — по одному на каждую цветовую модель:

```
const state = {
    rgb:  { r: 13, g: 200, b: 151 },
    cmyk: { c: 93.5, m: 0, y: 24.5, k: 21.569 },
    hsl:  { h: 164.28, s: 87.793, l: 41.765 }
};
```

Обновление состояния выполняется через три функции, соответствующие модели-источнику изменения:

```
function changeFromRGB(r, g, b) {
    state.rgb = { r, g, b };
    state.cmyk = rgbToCmyk(r, g, b);
    state.hsl = rgbToHsl(r, g, b);
    render();
}

function changeFromCMYK(c, m, y, k) {
    state.cmyk = { c, m, y, k };
    const rgb = cmykToRgb(c, m, y, k);
    state.rgb = rgb;
    state.hsl = rgbToHsl(rgb.r, rgb.g, rgb.b);
    render();
}

function changeFromHSL(h, s, l) {
    state.hsl = { h, s, l };
```

```
    const rgb = hslToRgb(h, s, l);
    state.rgb = rgb;
    state.cmyk = rgbToCmyk(rgb.r, rgb.g, rgb.b);
    render();
}
```

Ключевое правило: исходная модель сохраняется в состоянии *как есть*, без обратного пересчёта. Остальные модели рассчитываются как производные. Функция `render()` не выполняет никаких преобразований — она только раскладывает готовые значения из `state` в поля ввода, ползунки и на холсты.

4.3. Паттерн «Наблюдатель»

Описанная схема является вариантом паттерна *Observer*: каждая цветовая модель выступает и как *субъект* (хранит своё состояние и инициирует обновление), и как *наблюдатель* (пересчитывается при изменении других моделей). Изменение одной модели рассылает уведомление остальным, но не затирает собственные координаты источника. Это устраняет циклическую деградацию данных RGB $\rightarrow$ CMYK $\rightarrow$ RGB и делает поведение приложения корректным во всех крайних точках.

5. Пользовательский интерфейс

Интерфейс приложения включает:

- **Область предпросмотра** — крупный образец текущего цвета, HEX-код и системный выбор цвета браузера (`<input type="color">`).
- **Три карточки моделей** (RGB, CMYK, HSL). Каждая карточка содержит:
  - интерактивную палитру на элементе `<canvas>`, поддерживающую клик и перетаскивание;
  - ползунки для каждой компоненты модели;
  - текстовые поля точного числового ввода.

- **Блок «Готовый код»**, в котором в реальном времени отображается текущий цвет в формате HEX, Java, .NET, Swift, Objective-C и OpenGL.
- **Кнопки «Сброс»** и **«Копировать HEX»**.

Таким образом, для каждой из трёх моделей реализовано три независимых способа задания цвета: палитра, ползунок, текстовое поле. Это соответствует требованиям лабораторной работы.

5.1. Отрисовка палитр

Каждая палитра отрисовывается прямым заполнением массива пикселей, полученного через `ctx.createImageData(w, h)`:

- **RGB-палитра**: по оси X — компонента R, по оси Y — компонента G (инвертированная), компонента B фиксирована. Цвет пикселя задаётся напрямую без преобразований.
- **CMYK-палитра**: по осям X и Y — компоненты C и M, компоненты Y и K фиксированы. Цвет пикселя получается через `cmykToRgb`.
- **HSL-палитра**: кольцо (спектр) плюс вписанный треугольник. Угол на кольце соответствует оттенку H; внутри треугольника интерполируются S и L через барицентрические координаты.

6. Возникшие проблемы и их решения

6.1. Проблема необратимости преобразований

Как уже отмечалось в разделе об архитектуре, изначальная схема с единственным состоянием в RGB приводила к «залипанию» ползунков CMYK при $K = 100$ и ползунков HSL при $L = 0$ или $L = 100$. Решение — три независимых состояния и три функции изменения, сохраняющие исходные координаты без обратного пересчёта.

6.2. Дёргание маркера на HSL-палитре

На границах HSL-треугольника позиция маркера текущего цвета заметно «прыгала». Причина — двойное преобразование HSL → RGB (целые) → HSL при каждом обновлении. Ошибки округления при переходе к целочисленному RGB накапливались, а у границ треугольника, где насыщенность вычисляется через отношение малых разностей каналов, даже единичное расхождение в канале даёт скачок S и L на десятки процентов.

Решение также опирается на раздельное хранение состояний: при работе пользователя внутри HSL-палитры маркер строится напрямую из `state.hsl`, а не через `rgbToHsl(state.rgb)`.

6.3. Согласование отображения после пересчёта

После разделения состояний потребовалось убедиться, что поля ввода и ползунки всегда берут значения именно из соответствующей модели состояния (`state.cmyk` для CMYK-полей, `state.hsl` для HSL-полей), а не из повторного преобразования RGB. Это устранило визуальное «отскакивание» элементов интерфейса.

7. Выводы

В ходе лабораторной работы были изучены три ключевые цветовые модели — RGB, CMYK и HSL — и реализованы взаимные преобразования между ними. Было показано, что модели отражают разные аспекты восприятия и воспроизведения цвета: RGB описывает аддитивное смешение излучений (экраны), CMYK — субтрактивное смешение красок (печать), а HSL является перцептивной переупаковкой RGB, удобной для интерактивного выбора цвета.

Практическая часть работы показала, что основная сложность заключается не столько в математике преобразований, сколько в **архитектуре хранения состояния**. Попытка свести всё к единственной модели (RGB) приводит к необратимым потерям информации в особых точках (чистый чёрный в CMYK, крайние значения светлоты в HSL) и к накоплению ошибок округления при перетаскивании маркеров на палитре.

Применение подхода с тремя независимыми состояниями и функциями изменения от модели-источника, соответствующего паттерну «Наблюдатель», позволило:

- сохранять координаты, заданные пользователем в исходной модели, без их искажения;
- корректно обрабатывать крайние случаи ($K = 100$, $L = 0$, $L = 100$);
- уменьшить «дёргание» маркера на HSL-палитре;
- обеспечить стабильную и предсказуемую синхронизацию всех представлений цвета при любом способе ввода.

Приложение реализовано на чистом HTML5, CSS3 и JavaScript (ES6) без внешних библиотек, что позволило глубже разобраться в работе браузерных API: `CanvasRenderingContext2D`, обработке событий мыши и элементов управления формами.